

LAB TASK#9

NAME: MASHAAL ISHTIAQ
ROLL#: 22P-9326
SECTION:BCS-4C

CODE:

```
#include <iostream>
using namespace std;

class Node
{

private:
string name;
string department;
int id;
Node *left;
Node *right;

public:
Node()
{
name = " ";
department = " ";
id = 0;
left = NULL;
right = NULL;
}

Node(int id, string name, string department)
{
this->id = id;
this->name = name;
this->department = department;
left = NULL;
right = NULL;
}
friend class Employee;
friend int main();
};

class Employee
{

private:
Node *root;

public:
Employee()
{
root = NULL;
}
```

```
//_______Insert_____

Node *insert(Node *node, int id, string name, string department)
{

if (node == NULL)
{
Node *temp = new Node(id, name, department);

return temp;
}

else
{
if (node->id == id)
{
cout << "Already Exist" << endl;
return node;
}
else if (id < node->id)
node->left = insert(node->left, id, name, department);
else
{
node->right = insert(node->right, id, name, department);
}
return node;
}
}

void Insert(const int &id, const string &name, const string &department)
{

root = insert(root, id, name, department);

cout << "Employee has been successfully inserted" << endl;
}

//______Search ______

Node *Search(int id)
{

return search(root, id);
}

Node *search(Node *node, int id)
{
if (node == NULL)
{
return NULL;
}
else if (node->id == id)
```

```
return node;

else if (id < node->id)

return search(node->left, id); // recursion

else
return search(node->right, id);
}

//_____Display____

void Display(Node *root)
{

if (root == NULL)
{
cout << "No employees to display." << endl;
return;
}

if (root->left != NULL)
{
Display(root->left);
}

cout << "Employee ID: " << root->id << ", Name: " << root->name << ", Department: " << root-
>department << endl;

if (root->right != NULL)
{
Display(root->right);
}
}

//___Find Same Authority(sibling)____

Node *FindSameAuthority(int id)
{
return findSameAuthority(root, id);
}

Node *findSameAuthority(Node *root, const int id)
{
Node *temp = search(root, id);

if (root == NULL)
{
return NULL;
}
if (root->left != NULL && root->left->id == id)
{
```

```
return root->right;
}
else if (root->right != NULL && root->right->id == id)
{
return root->left;
}

if (id > root->id)
{
return findSameAuthority(root->right, id);
}
else
{
return findSameAuthority(root->left, id);
}
}

//________Total Depth________

int TotalDepth()
{
return totalDepth(root);
}

int totalDepth(Node *root)
{
if (root == NULL)
{
return 0;
}
int LeftDepth = totalDepth(root->left);
int RightDepth = totalDepth(root->right);
return max(LeftDepth, RightDepth) + 1; // adding the root node as well so-> +1
}

//_____Find the Subordinate(parent)_____

Node* FindSubordinate(int id)
{

return findSubordinate(root, id);
}

Node *findSubordinate(Node *root, int id)
{
Node *temp = search(root, id);
if (root == NULL)
{
return NULL;
}
if (root->left != NULL && root->left->id == id)
{
```

```
return root;
}
else if (root->right != NULL && root->right->id == id)
{
return root;
}

if (id > root->id)
{
return findSubordinate(root->right, id);
}
else
{
return findSubordinate(root->left, id);
}
}
friend int main();
};

int main()
{
Employee emp;

int choice, id;
string name, department;
Node *ptr;
while (1)
{
cout << "\n______Employee Management System_______\n"
<< "1. Insert an employee\n"
<< "2. Search for an employee\n"
<< "3. View all employee details\n"
<< "4 Find employees with same authority\n"
<< "5. Total depth of employee hierarchy\n"
<< "6. Find Subordinates of employees\n"
<< "7. Exit\n"
<< "Enter your choice: ";
cin >> choice;

switch (choice)
{
case 1:
cout << "\nEnter employee id: ";
cin >> id;

cout << "\nEnter employee name: ";
cin.ignore();
getline(cin, name);

cout << "\nEnter employee department: ";
cin.ignore();
getline(cin, department);
```

```
emp.Insert(id, name, department);
break;

case 2:
cout << "\nEnter employee id to search: ";

cin >> id;

ptr = emp.Search(id);
if (ptr != NULL)
{
cout << "Employee Found!" << endl;
cout << "Employee id: " << ptr->id << endl;
cout << "Employee name: " << ptr->name << endl;
cout << "Employee department: " << ptr->department << endl;
}
else
{
cout << "Employee with the ID " << id << " not found!" << endl;
}

break;

case 3:

cout << "\nEmployee details: " << endl;
emp.Display(emp.root);
break;

case 4:

cout << "\nEnter Employee id: " << endl;
cin >> id;
ptr = emp.FindSameAuthority(id);
cout << "The employee with same Authority is " << ptr->id << endl;

// find(root, department);
break;

case 5:
// depth(root);
cout << "Total depth of the employee hierarchy: " << emp.TotalDepth() << endl;

break;

case 6:

cout << "\nEnter Employee id: " << endl;
cin >> id;
ptr= emp.FindSubordinate(id);
cout << "The employee subordinate is " << ptr->id << endl;
```

```
break;

case 7:
cout << "Exiting..." << endl;
return 0;

default:
cout << "\nInvalid choice";
break;
}
}
}
```

```
Ishtiaq_task#9.cpp -o p229326_mashaalIshtiaq

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 1

Enter employee id: 5

Enter employee name: a

Enter employee department: a
Employee has been successfully inserted

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 1

Enter employee id: 3
```

```
Enter employee department: b
Employee has been successfully inserted

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 1

Enter employee id: 10

Enter employee name: c

Enter employee department: c
Employee has been successfully inserted

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 1
```

```
Enter employee department: d
Employee has been successfully inserted

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 2

Enter employee id to search: 3
Employee Found!
Employee id: 3
Employee name: b
Employee department:

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 4
```

```
Enter your choice: 6

Enter Employee id:
3

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 3

Employee details:
Employee ID: 2, Name: d, Department:
Employee ID: 3, Name: b, Department:
Employee ID: 5, Name: a, Department:
Employee ID: 10, Name: c, Department:

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice:
```

```
7. Exit
Enter your choice: 4

Enter Employee id:
3
The employee with same Authority is  10

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 6

Enter Employee id:
3
The employee subordinate is  5

______Employee Management System_______
1. Insert an employee
2. Search for an employee
3. View all employee details
4  Find employees with same authority
5. Total depth of employee hierarchy
6. Find Subordinates of employees
7. Exit
Enter your choice: 7
Exiting...
mashaal@mashaal-Latitude-7480:~/Desktop/sems
```